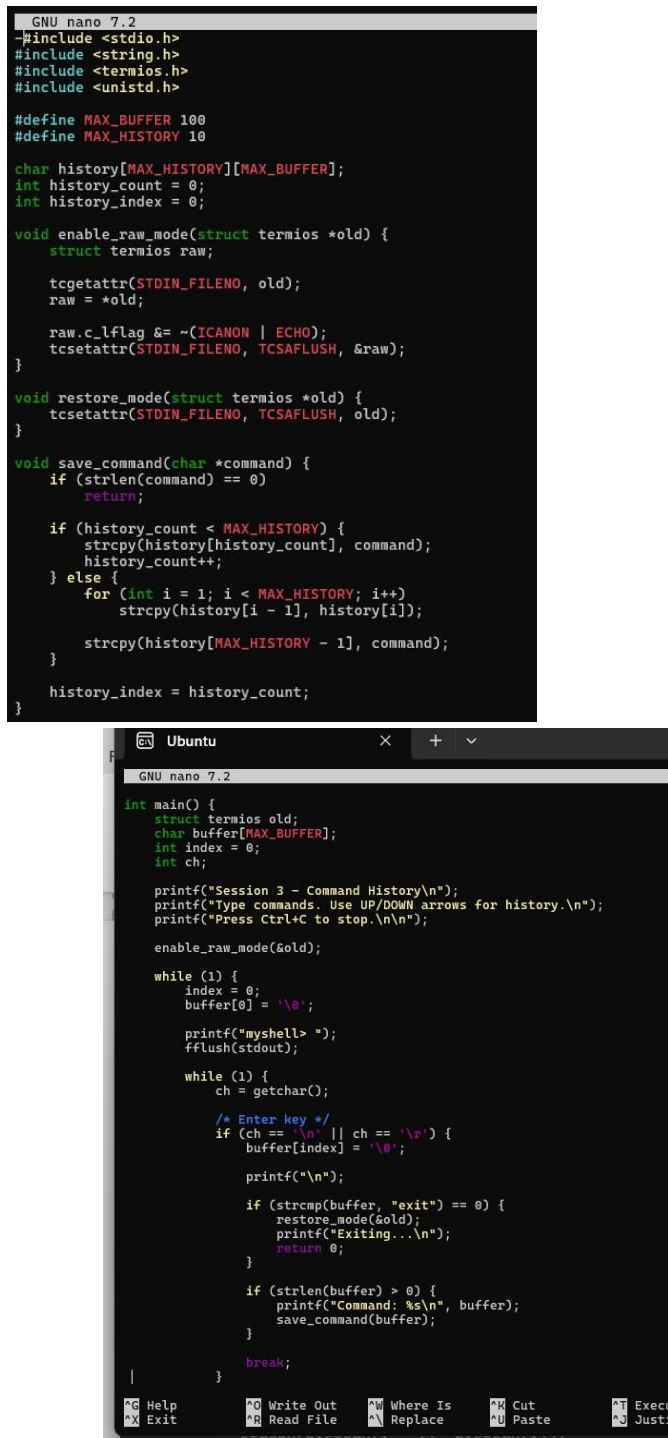


SKILL WEEK-2

Session3:

Task1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

Source code:



```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_BUFFER 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_BUFFER];
int history_count = 0;
int history_index = 0;

void enable_raw_mode(struct termios *old) {
    struct termios raw;

    tcgetattr(STDIN_FILENO, old);
    raw = *old;

    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void restore_mode(struct termios *old) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, old);
}

void save_command(char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_index = history_count;
}

int main() {
    struct termios old;
    char buffer[MAX_BUFFER];
    int index = 0;
    int ch;

    printf("Session 3 - Command History\n");
    printf("Type commands. Use UP/DOWN arrows for history.\n");
    printf("Press Ctrl+C to stop.\n\n");

    enable_raw_mode(&old);

    while (1) {
        index = 0;
        buffer[0] = '\0';

        printf("myshell> ");
        fflush(stdout);

        while (1) {
            ch = getchar();

            /* Enter key */
            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';

                printf("\n");

                if (strcmp(buffer, "exit") == 0) {
                    restore_mode(&old);
                    printf("Exiting...\n");
                    return 0;
                }

                if (strlen(buffer) > 0) {
                    printf("Command: %s\n", buffer);
                    save_command(buffer);
                }

                break;
            }
        }
    }
}
```

```

GNU nano 7.2

/* Backspace */
if (ch == 127 || ch == 8) {
    if (index > 0) {
        index--;
        buffer[index] = '\0';

        printf("\b\b");
        fflush(stdout);
    }
    continue;
}

/* Escape sequence */
if (ch == 27) {
    int ch2 = getchar();

    if (ch2 == '[') {
        int ch3 = getchar();

        /* UP arrow */
        if (ch3 == 'A') {
            if (history_index > 0) {
                history_index--;

                while (index > 0) {
                    printf("\b\b");
                    index--;
                }

                strcpy(buffer, history[history_index]);
                index = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        /* DOWN arrow */
        else if (ch3 == 'B') {
            if (history_index < history_count - 1) {
                history_index++;
            }
        }
    }
}

/* Normal character */
if (index < MAX_BUFFER - 1) {
    buffer[index++] = ch;
    buffer[index] = '\0';

    putchar(ch);
    fflush(stdout);
}

restore_mode(sold);
return 0;

```

```

Ubuntu
GNU nano 7.2

/* DOWN arrow */
else if (ch3 == 'B') {
    if (history_index < history_count - 1) {
        history_index++;

        while (index > 0) {
            printf("\b\b");
            index--;
        }

        strcpy(buffer, history[history_index]);
        index = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    } else {
        history_index = history_count;
    }

    while (index > 0) {
        printf("\b\b");
        index--;
    }

    buffer[0] = '\0';
}

}

continue;
}

/* Normal character */
if (index < MAX_BUFFER - 1) {
    buffer[index++] = ch;
    buffer[index] = '\0';

    putchar(ch);
    fflush(stdout);
}

}

restore_mode(sold);
return 0;

```

Output:

```

avulasanjanana@SanjanaReddy:~$ nano session3_task1.c
avulasanjanana@SanjanaReddy:~$ ^C
avulasanjanana@SanjanaReddy:~$ gcc session3_task1.c -o session3_task1
avulasanjanana@SanjanaReddy:~$ ./session3_task1
Session 3 - Command History
Type commands. Use UP/DOWN arrows for history.
Press Ctrl+C to stop.

myshell> hello
You entered: hello
myshell> test
You entered: test

```

Observation: I learned how to use escape sequences to detect special keyboard inputs such as the Up and Down arrow keys. I learned how to store previously entered commands in a history buffer and navigate through them using the arrow keys. I also learned how to update the input buffer when recalling previous commands and manage user input in an interactive shell.

Task2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    char *buffer;
    size_t size = INITIAL_SIZE;
    size_t length = 0;
    int ch;

    Node *head = NULL;
    Node *tail = NULL;

    buffer = malloc(size);

    if (buffer == NULL) {
        perror("malloc failed");
        return 1;
    }

    printf("Session 3 - Dynamic Memory Management\n");
    printf("Enter commands. Type 'exit' to finish.\n\n");

    while (1) {
        length = 0;

        printf("myshell> ");
        fflush(stdout);

```

```

GNU nano 7.2
        while ((ch = getchar()) != '\n' && ch != EOF) {

            if (length + 1 >= size) {
                size *= 2;

                char *temp = realloc(buffer, size);

                if (temp == NULL) {
                    perror("realloc failed");
                    free(buffer);
                    return 1;
                }

                buffer = temp;
            }

            buffer[length++] = ch;

            buffer[length] = '\0';

            if (strcmp(buffer, "exit") == 0) {
                break;
            }

            if (length == 0) {
                continue;
            }

            Node *new_node = malloc(sizeof(Node));

            if (new_node == NULL) {
                perror("malloc failed");
                break;
            }

```

```
GNU nano 7.2

new_node->command = malloc(length + 1);
if (new_node->command == NULL) {
    perror("malloc failed");
    free(new_node);
    break;
}

strcpy(new_node->command, buffer);
new_node->next = NULL;

if (head == NULL) {
    head = new_node;
    tail = new_node;
} else {
    tail->next = new_node;
    tail = new_node;
}

printf("Stored command: %s\n", buffer);
printf("Buffer size: %zu bytes\n", size);
}

free(buffer);
Node *current = head;
while (current != NULL) {
    Node *next = current->next;
    free(current->command);
    free(current);
    current = next;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^_ Replace    ^U Paste
```

```
        current = next;
    }

    printf("\nAll dynamically allocated memory released.\n");

    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^_ Replace    ^U Paste
```

Output:

```
avulasanjana@SanjanaReddy:~$ nano session3_task2.c
avulasanjana@SanjanaReddy:~$ gcc session3_task2.c -o session3_task2
avulasanjana@SanjanaReddy:~$ ./session3_task2
Session 3 - Dynamic Memory Management
Enter commands. Type 'exit' to finish.
myshell> heloo
Stored command: heloo
Buffer size: 10 bytes
myshell> exit
All dynamically allocated memory released.
```

Observation: I learned how to use dynamic memory allocation in C using malloc() and realloc(). I learned how to resize a buffer when more memory is required and how to store commands using a linked list. I also learned the importance of releasing dynamically allocated memory using free() to prevent memory leaks.

Session4

Task1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 50

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} token;

int main() {
    char input[500];
    token tokens[MAX_TOKENS];
    int token_count = 0;

    printf("Session 4 - Tokenization\n");
    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        /* Skip whitespace */
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n')
            break;

        /* Check token limit */
        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }

        int j = 0;
        /* Read characters until whitespace or delimiter */
        while (input[i] != '\0' &&
            input[i] != '\n' &&
            !isspace((unsigned char)input[i]) &&
            input[i] != '|' &&
            input[i] != '>') {
            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count].value[j++] = input[i];
            }
            i++;
        }
    }
}
```

```
tokens[token_count].value[j] = '\0';
if (j > 0) {
    token_count++;
}

/* Handle delimiters */
if (input[i] == '|' || input[i] == '>') {
    if (token_count >= MAX_TOKENS) {
        printf("Error: Too many tokens.\n");
        break;
    }
    tokens[token_count].value[0] = input[i];
    tokens[token_count].value[1] = '\0';
    token_count++;
    i++;
}

printf("\nTokens identified:\n");

if (token_count == 0) {
    printf("No tokens found.\n");
    return 0;
}

for (int k = 0; k < token_count; k++) {
    printf("Token %d: %s\n", k + 1, tokens[k].value);
}

printf("\nToken stream validation:\n");

int valid = 1;
for (int k = 0; k < token_count; k++) {
    if (strlen(tokens[k].value) == 0) {
        valid = 0;
    }
}

if (valid)
    printf("Token stream is valid.\n");
else
    printf("Token stream is invalid.\n");

return 0;
}
```

Output:

```
avulasanjana@SanjanaReddy:~$ nano session4_task1.c
avulasanjana@SanjanaReddy:~$ gcc session4_task1.c -o session4_task1
avulasanjana@SanjanaReddy:~$ ./session4_task1
Session 4 - Tokenization
Enter a command: ls -l | grep txt

Tokens identified:
Token 1: [ls]
Token 2: [-l]
Token 3: [grep]
Token 4: [txt]

Token stream validation:
Token stream is valid.
avulasanjana@SanjanaReddy:~$ |
```

Observation: I learned how to split a command into individual tokens using tokenization. I understood how whitespace and special delimiters such as | can separate different parts of a command. I also learned how to store tokens in a token structure and validate the resulting token stream before further processing.

Task2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50

int main() {
    char input[50];
    char *tokens[MAX_TOKENS];
    int token_count = 0;
    int valid = 1;

    printf("Session 4 - Parser\n");
    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command detected.\n");
        return 0;
    }

    /* Tokenize input */
    char *token = strtok(input, " \t");

    while (token != NULL && token_count < MAX_TOKENS) {
        tokens[token_count++] = token;
        token = strtok(NULL, " \t");
    }

    printf("\nParse Structure:\n");

    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

```

```

    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    /* Basic syntax validation:
     * A pipe cannot be the first or last token.
     * Two pipes cannot occur consecutively.
     */
    for (int i = 0; i < token_count; i++) {
        if (strcmp(tokens[i], "|") == 0) {
            if (i == 0 || i == token_count - 1) {
                valid = 0;
                printf("Syntax Error: Invalid pipe position.\n");
            }

            if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
                valid = 0;
                printf("Syntax Error: Consecutive pipes detected.\n");
            }
        }
    }

    if (!valid) {
        printf("\nParsing failed.\n");
        return 1;
    }

    /* Display execution structure */
    printf("Command: %s\n", tokens[0]);

    if (token_count > 1) {
        printf("Arguments:\n");

```

```

        if (token_count > 1) {
            printf("Arguments:\n");

            for (int i = 1; i < token_count; i++) {
                if (strcmp(tokens[i], "|") != 0) {
                    printf("  Argument %d: %s\n", i, tokens[i]);
                }
            }
        }

        printf("\nParse tree / execution structure:\n");
        printf("ROOT\n");
        printf("└─ COMMAND: %s\n", tokens[0]);

        for (int i = 1; i < token_count; i++) {
            if (strcmp(tokens[i], "|") == 0) {
                printf("    └─ PIPE\n");
            } else {
                printf("    └─ ARGUMENT: %s\n", tokens[i]);
            }
        }

        printf("\nSyntax validation: SUCCESS\n");
        printf("Execution structure generated successfully.\n");

        return 0;
    }
}

```

Output:

```

avulasanjana@SanjanaReddy:~$ nano session4_task2.c
avulasanjana@SanjanaReddy:~$ gcc session4_task2.c -o session4_task2
avulasanjana@SanjanaReddy:~$ ./session4_task2
Session 4 - Parser
Enter a command: ls -l | grep txt

Parse Structure:
Command: ls

Command: ls
Arguments:
  Argument 1: -l
  Argument 3: grep
  Argument 4: txt

Parse tree / execution structure:
ROOT
└─ COMMAND: ls
   └─ ARGUMENT: -l
      └─ PIPE
         └─ ARGUMENT: grep
            └─ ARGUMENT: txt

Syntax validation: SUCCESS
Execution structure generated successfully.
avulasanjana@SanjanaReddy:~$

```

Observation: I learned how to design basic parser logic for processing user commands. I understood how to validate command syntax, detect errors such as invalid or consecutive pipe symbols, and handle empty commands. I also learned how tokens can be organized into a parse structure containing commands, arguments, and pipes, which can later be used for command execution.